Och

1 Introduction

After playing around with dependent types in an imperative programming setting during the masters thesis, I came to the conclusion that a fairly uncommon combination of language features would lead to an excellent combination of developer ergonomics and expressivity.

I want to study the most unusual + unweildly of these features in isolation with a core calculus before building full Ochre, specifically:

- The "terms are types" philosophy, as exonerated by PSS [1]. In Och this comes out roughly to "types are terms with holes".
- Terms/types form a subtyping lattice, where supertypes can be freely substituted for any subtype without any coercions.

TODO: get rid of the "but why are these features important" feeling

1.1 Goals

Soundness - Ochre is only valuable if they can trust that the compiler will catch runtime errors at compile time. Proving the soundness of systems with the above features, let alone ownership + mutation is new research, Och is a place to do that research.

Expressivity - The type system must be able to articulate properties and catch bugs that people care about. E.g. it must be expressive enough to tell the difference between $x + y$ and $x + x$ if one is correct and the other incorrect.

Extensibility - Och is the first in a series of languages which will eventually culminate in full Ochre. As such, there is no point working with features which won't be re-usable. This has already manifested in the decision to tackle arbitrary unbounded recursion via fix instead of having to add primitive inductive types and only well-founded induction.

2 Related Work

Pure Subtype Systems [1]

3 Language Semantics

The term language is

$$
\begin{array}{rr}
e, \tau ::= x & \text{variable} \\
\mid \lambda(x \leq \tau).e & \text{lambda} \\
\mid e_1\ e_2 & \text{application} \\
\mid \top & \text{universe (top)} \\
\mid \bot & \text{primitive bottom} \\
\mid \iota(x \leq \tau).e & \text{self-type binder} \\
\mid \text{fix}(x \leq \tau).e & \text{recursive binder}
\end{array}
$$

There is no separate type category: every τ above is itself a term. Throughout the paper Γ denotes a typing context (a finite map from variable names to their declared types).

Variables, lambdas, application as per the standard λ-calculus, except λ has a domain type annotation which is respected by the type checker.

$\top$ **and** $\bot$ are the top and bottom of the subtyping lattice. $\top$ represents the widest possible type, which tells you nothing about the term being typed, and $\bot$ is the empty type.

Fix $\text{fix}(x \le \tau).e$ allows a term to be defined in terms of itself. x is the reference to self, and τ is the "upper bound" which the whole expression can be assumed to have while it's being type checked (which prevents loops during type checking).

Iota $\iota(x \le \tau).e$ allows a type to be defined in terms of **the term inhabiting it**. This means $e \sqsubseteq \iota(x : \tau_0).\tau_1$ is reduced to $e \sqsubseteq \tau_1[x \mapsto e]$ during checking (notice: the LHS has moved to the RHS). This is a Cedille-style self type [2], and allows for church encodings with dependent elimination instead of having to add datatypes to the language as a primitive.

3.1 Concrete semantics — the evaluation judgment

The concrete evaluator is a substitution-based call-by-value big-step interpreter on closed terms. Lambdas, ι, and fix are values; applications in function position unroll the recursive binder by substituting its self-reference, then re-apply.

Concrete evaluation is *supposed* to represent Och's runtime semantics, but Och has no concept of levels/universes/stages, so the type checker cannot enforce that $\{\top, \bot, \iota\}$ don't turn up at runtime. Since soundness is defined roughly as "terms accepted by the type checker never crash at runtime", we must handle these values in the concrete semantics to avoid making our soundness completely unprovable.

This can and should be solved in the future by adding levels to all binders (see §3.6 "Adding Universes" of [1] for roughly how I plan on doing this).

We write the judgment $e \Downarrow v$ for "closed term e concretely evaluates to value v."

$$\frac{}{v \Downarrow v}\ \text{E-Val} \qquad \frac{f \Downarrow \lambda(x \le \tau).b \quad a \Downarrow v_a \quad b[x \mapsto v_a] \Downarrow v}{f\ a \Downarrow v}\ \text{E-App}$$

$$\frac{f \Downarrow \iota(x \le \tau).b \quad a \Downarrow v_a \quad (b[x \mapsto \iota(x \le \tau).b])\ v_a \Downarrow v}{f\ a \Downarrow v}\ \text{E-App-Iota} \qquad \frac{f \Downarrow \text{fix}(x \le \tau).b \quad a \Downarrow v_a \quad (b[x \mapsto \text{fix}(x \le \tau).b])\ v_a \Downarrow v}{f\ a \Downarrow v}\ \text{E-App-fix}$$

$$\begin{aligned} \text{where } v \in \text{Value} ::= {} & \lambda(x \le \tau).e && \text{lambda} \\ \mid {} & \bot && \text{primitive bottom} \\ \mid {} & \top && \text{universe (top)} \\ \mid {} & \iota(x \le \tau).e && \text{self-type binder} \\ \mid {} & \text{fix}(x \le \tau).e && \text{recursive binder} \end{aligned}$$

Free variables are not values.

$\bot$ is a self-evaluating value. Like $\top$, $\bot$ evaluates to itself via [E-Val] and has no application dispatch arm, so applying an argument to $\bot$ is *stuck* (parallel to $\top$-application). This is intentional: $\bot$ is a type, not a callable. The typing discipline must ensure well-typed programs never reach $\bot\ a$ at runtime.

This is the **operational specification** of the language.

4 Typing Rules

Och's type system is structured around two relations:

Well-formedness $\Gamma \vdash e\ \text{wf}$ — is e well-formed under Γ? Defined by structural rules (Section 4.3), delegating all type-comparison questions to subtyping. No type is assigned; in Och, a value IS its own type (by [S-Refl]).

Subtyping $S; \Gamma \vdash a \sqsubseteq b$ — is a a subtype of b under Γ, assuming coinductive hypotheses S? Subtyping in Och means set inclusion: $A \sqsubseteq B$ iff every value of A is also a value of B.

4.1 Context Γ

Γ is the typing context — a finite ordered list of $(\text{variable}, \text{type})$ pairs, with the most recent binder at the front. Lookup is $\Gamma(x)$ for the declared type of x; extension is $\Gamma, x \le A$.

4.2 Seen set S

S is a set of pairs (a, b) — ancestor subtyping goals introduced by productive unfolding rules (ι-introduction and the four unfold rules, Section 4.7). The invariant: only those five rules extend S; every other rule propagates it unchanged. This is the Brandt–Henglein device [3] for equirecursive subtyping — coinductive assumptions are only legal after at least one productive step, so reflexivity of a non-productive goal cannot be closed by a hypothesis. In effect, S is a finite representation of a coinductive (potentially infinite) proof tree: when a goal recurs, the branch closes rather than recursing forever, encoding a regular infinite derivation as a finite one.

The "real" subtyping judgment is $\emptyset; \Gamma \vdash a \sqsubseteq b$ (empty hypothesis set); non-empty S arises only inside a derivation.

4.3 Well-formedness

The well-formedness judgment $\Gamma \vdash e$ wf determines whether term e is well-formed under context Γ. It does not assign a type — in Och's "terms are types" world, a value IS its own most-precise type (by [S-Refl]), and all type-comparison questions are handled by subtyping. Well-formedness validates purely structural conditions: annotations are types, binders are in scope, and applications have Π-typed functions with domain-inhabiting arguments.

$$\frac{x \in \mathrm{dom}(\Gamma)}{\Gamma \vdash x \text{ wf}}\ \text{W-Var} \qquad \frac{}{\Gamma \vdash \top \text{ wf}}\ \text{W-Type} \qquad \frac{}{\Gamma \vdash \bot \text{ wf}}\ \text{W-Bot}$$

$$\frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \le A \vdash b \text{ wf}}{\Gamma \vdash \lambda(x \le A).b \text{ wf}}\ \text{W-Lam} \qquad \frac{\Gamma \vdash f \text{ wf} \quad \Gamma \vdash a \text{ wf} \quad \emptyset; \Gamma \vdash f \sqsubseteq \lambda(x \le A).B \quad \emptyset; \Gamma \vdash a \sqsubseteq A}{\Gamma \vdash f\ a \text{ wf}}\ \text{W-App}$$

$$\frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \le A \vdash b \text{ wf}}{\Gamma \vdash \iota(x \le A).b \text{ wf}}\ \text{W-Iota} \qquad \frac{\Gamma \vdash A \text{ wf} \quad \Gamma, x \le A \vdash b \text{ wf}}{\Gamma \vdash \mathrm{fix}(x \le A).b \text{ wf}}\ \text{W-Fix}$$

[W-Var] requires the variable to be in scope.

[W-Type] and **[W-Bot]** are always well-formed.

[W-Lam], [W-Iota], [W-Fix] check that the annotation is well-formed and that the body is well-formed under the extended context.

[W-App] is the only rule with subtyping premises: the function must subtype some $\lambda(x \le A).B$ (it has a Π-type), and the argument must subtype the domain A. No type is assigned to the result — the result's type is the result itself (by [S-Refl]), and if the caller needs to know it subtypes something, that is a separate subtyping question.

4.4 Rule taxonomy

The rules fall into four categories. Each serves a distinct purpose; knowing which category a rule belongs to predicts its shape.

Category	Purpose	Rules	Extends S?
Structural (Section 4.5)	Plumbing: compose, lookup, without inspecting constructors on either side	S-Refl, S-Top, S-BotL, S-Trans, S-Hyp, S-Var	no
Congruence (Section 4.6)	Match constructor on both sides; reduce to sub-obligations with variance	S-Lam, S-App-Cong, S-Iota-Cong, S-Fix-Cong	no
Productive unfolding (Section 4.7)	Unfold a recursive binder (ι/fix); extend S; enable coinductive closure	S-Iota-Intro, S-Unfold-Iota-L/R, S-Unfold-Fix-L/R	**yes**
Conversion (Section 4.8)	Close under head reduction so subtyping is closed under computation	S-Beta-L/R	no

Table 1: Rule taxonomy for declarative subtyping

The S-extension column matters: S-Hyp can only fire against entries that some ancestor productive rule installed, so any path to S-Hyp must cross an unfold — the productivity requirement made mechanical.

4.5 Structural rules

"Structural" rules talk about $\sqsubseteq$ as a relation on terms without inspecting either side's head constructor: reflexivity, transitivity, the top element, hypothesis lookup, and variable lookup.

$$\frac{}{S;\Gamma \vdash e \sqsubseteq e}\ \text{S-Refl} \qquad \frac{}{S;\Gamma \vdash e \sqsubseteq \top}\ \text{S-Top} \qquad \frac{}{S;\Gamma \vdash \bot \sqsubseteq e}\ \text{S-BotL}$$

$$\frac{S;\Gamma \vdash a \sqsubseteq b \quad S;\Gamma \vdash b \sqsubseteq c}{S;\Gamma \vdash a \sqsubseteq c}\ \text{S-Trans} \qquad \frac{(a,b) \in S}{S;\Gamma \vdash a \sqsubseteq b}\ \text{S-Hyp} \qquad \frac{}{S;\Gamma \vdash x \sqsubseteq \Gamma(x)}\ \text{S-Var}$$

Ex falso via subsumption. There is no dedicated "absurd" eliminator. If $a \sqsubseteq \bot$ is derivable, then $a \sqsubseteq e$ for every e via [S-Trans] on [S-BotL]. The "contradiction" discharge is subsumption alone. This matches the DOT tradition: $\bot$ inhabits every type trivially in subtyping, so any term whose type is already $\bot$ flows into any expected type without further ceremony.

Why [S-Trans] is a constructor, not a derived theorem. In a normal simply-typed subtyping relation, transitivity is admissible: a standard induction on the shape of the two derivations composes them. That argument requires a decreasing syntactic measure — typically, both derivations get strictly smaller in each case of the composition. Och's four unfold rules and iota-intro break this: unfolding $\text{fix}(x \le A).\ \text{body}$ replaces it with $\text{body}[x \mapsto \text{fix}(x \le A).\ \text{body}]$, which is *larger* than the original. There's no obvious structural measure on derivations that decreases through an unfold, so transitivity is not derivable by induction on derivations. The seen-set discipline of Section 4.7 is the coinductive counterpart that keeps the relation consistent, but it doesn't by itself give admissibility of transitivity.

Eliminating transitivity as a primitive rule (*transitivity elimination*) is highly desirable: it simplifies metatheory and makes the relation syntax-directed. Hutchins' Pure Subtype Systems [1] — a closely related system — did not manage to eliminate transitivity. Pasquale and García-Pérez [4] succeeded in a continuation of that work, but required switching the entire theory to a Krivine machine.

4.6 Congruence rules

Congruence rules *do* inspect the head constructor on both sides, require it to match, and reduce the goal to sub-obligations on the immediate sub-terms with appropriate variance.

$$\frac{S;\Gamma \vdash A_2 \sqsubseteq A_1 \qquad S;\Gamma, x \le A_2 \vdash b_1 \sqsubseteq b_2}{S;\Gamma \vdash \lambda(x \le A_1).b_1 \sqsubseteq \lambda(x \le A_2).b_2}\ \text{S-Lam} \qquad \frac{S;\Gamma \vdash f_2 \sqsubseteq f_1 \qquad S;\Gamma \vdash a_2 \sqsubseteq a_1}{S;\Gamma \vdash a_1 \sqsubseteq a_2}\ \text{S-App-Cong}$$

$$\frac{S;\Gamma \vdash A_1 \sqsubseteq A_2 \qquad S;\Gamma, x \le A_2 \vdash B_1 \sqsubseteq B_2}{S;\Gamma \vdash \iota(x \le A_1).B_1 \sqsubseteq \iota(x \le A_2).B_2}\ \text{S-Iota-Cong}$$

$$\frac{\begin{array}{c} S;\Gamma \vdash A_1 \sqsubseteq A_2 \\ S;\Gamma, x \le A_2 \vdash b_1 \sqsubseteq b_2 \end{array}}{S;\Gamma \vdash \mathrm{fix}(x \le A_1).b_1 \sqsubseteq \mathrm{fix}(x \le A_2).b_2}\ \text{S-Fix-Cong}$$

The equivalence premise on [S-App-Cong] (both directions on the argument) is necessary because a neutral head can use its argument at any variance, so equivalence is the only sound congruence.

4.7 Productive unfolding (extend S)

A rule is *productive* when it replaces a goal whose head is a recursive binder (ι or fix) with a goal where that binder has been unfolded once. Unfolding makes the term *larger* in syntactic size, so no structural induction can traverse an arbitrary chain of unfolds. Productivity instead provides a *coinductive* handle: once at least one unfold has fired, the original goal is guaranteed to eventually re-appear as an ancestor, at which point [S-Hyp] can close the derivation.

The rules below are the only ones that extend S. Let $S' = S \cup \{(a, b)\}$ where (a, b) is the current goal.

$$\frac{\begin{array}{c} S';\Gamma \vdash a \sqsubseteq A \\ S';\Gamma \vdash a \sqsubseteq B[x \mapsto a] \end{array}}{S;\Gamma \vdash a \sqsubseteq \iota(x \le A).B}\ \text{S-Iota-Intro} \qquad \frac{S';\Gamma \vdash B[x \mapsto \iota(x \le A).B] \sqsubseteq c}{S;\Gamma \vdash \iota(x \le A).B \sqsubseteq c}\ \text{S-Unfold-Iota-L}$$

$$\frac{S';\Gamma \vdash a \sqsubseteq B[x \mapsto \iota(x \le A).B]}{S;\Gamma \vdash a \sqsubseteq \iota(x \le A).B}\ \text{S-Unfold-Iota-R}$$

$$\frac{S';\Gamma \vdash b[x \mapsto \mathrm{fix}(x \le A).b] \sqsubseteq c}{S;\Gamma \vdash \mathrm{fix}(x \le A).b \sqsubseteq c}\ \text{S-Unfold-Fix-L} \qquad \frac{S';\Gamma \vdash a \sqsubseteq b[x \mapsto \mathrm{fix}(x \le A).b]}{S;\Gamma \vdash a \sqsubseteq \mathrm{fix}(x \le A).b}\ \text{S-Unfold-Fix-R}$$

S-Unfold-Iota-R is the weaker sibling of S-Iota-Intro (same conclusion, no annotation premise), needed to close the equivalence of ι-unfolding.

4.8 Conversion

The subtyping relation must be closed under head reduction: if a computes to a', then $a \sqsubseteq b$ should hold iff $a' \sqsubseteq b$. These rules provide that closure.

$$\frac{S;\Gamma \vdash \mathrm{body}[x \mapsto \mathrm{arg}] \sqsubseteq b}{S;\Gamma \vdash (\lambda(x \le A).\ \mathrm{body})\ \ \mathrm{arg} \sqsubseteq b}\ \text{S-Beta-L} \qquad \frac{S;\Gamma \vdash a \sqsubseteq \mathrm{body}[x \mapsto \mathrm{arg}]}{S;\Gamma \vdash a \sqsubseteq (\lambda(x \le A).\ \mathrm{body})\ \ \mathrm{arg}}\ \text{S-Beta-R}$$

5 Metatheory

Soundness connects the concrete semantics (Section 3.1), subtyping (Section 4), and well-formedness (Section 4.3).

Evaluation equivalence. If e evaluates to e', both directions of subtyping hold.

$$e \text{ closed} \wedge e \Downarrow e' \Rightarrow \emptyset;\emptyset \vdash e' \sqsubseteq e \wedge \emptyset;\emptyset \vdash e \sqsubseteq e'$$

Preservation. Evaluation preserves well-formedness (derives from evaluation equivalence).

$$\vdash e \text{ wf} \wedge e \Downarrow e' \Rightarrow \vdash e' \text{ wf}$$

Progress. A well-formed closed term doesn't get stuck during evaluation. The evaluator may return a value or run out of fuel but never reaches an error state.

End-to-end. Composing the above: if e is well-formed and subtypes τ, and e evaluates to e', the result subtypes τ.

$$\vdash e \text{ wf} \land \emptyset;\emptyset \vdash e \sqsubseteq \tau \land e \Downarrow e' \Rightarrow \emptyset;\emptyset \vdash e' \sqsubseteq \tau$$

5.1 What soundness promises to the programmer

Two separable runtime properties a type system can promise:

1. **Termination.** Running a well-typed program eventually produces a value. A totality / normalization claim.
2. **No runtime type errors.** Conditional on the program *not* diverging, the value it produces is well-formed and the program never reaches a stuck state.

Och **does not aim to prove (1).** Consistency and normalization are explicitly deferred; the calculus admits non-terminating terms by design (type-in-type; general recursion via fix).

Och **does aim to prove (2)**, and this is the real runtime guarantee of the type system. The language expects the following discipline: **type-check first; only evaluate if it succeeded.**

6 Appendix A. Lean Formalisation

The entire calculus described in this paper is formalised in Lean 4. This appendix maps the paper's named-variable presentation to the mechanised definitions, and notes the one systematic difference: the formalisation uses **de Bruijn indices** throughout, so named variables x become positional indices, substitution $\text{body}[x \mapsto v]$ becomes index arithmetic, and context lookup $\Gamma(x)$ becomes list indexing.

6.1 Representation

All terms, types, and values share a single inductive `Expr`. Named variables x are represented by `Expr.bvar` (a natural-number index); substitution uses `Expr.subst`/`Expr.shift` for standard de Bruijn arithmetic. The context Γ is `List Expr`, indexed by position with the innermost binder at index 0.

A named variable x bound at the innermost binder becomes `bvar 0`; a variable bound k binders out becomes `bvar k`; substitution $\text{body}[x \mapsto v]$ becomes `body.subst 0 v`.

6.2 Correspondence

Paper concept	Lean definition	File
Term syntax (e, τ)	`Expr`	`Syntax.lean`
Concrete evaluation $(e \Downarrow v)$	`concEval`	`Eval.lean`
Declarative subtyping $(S;\Gamma \vdash a \sqsubseteq b)$	`Subtype'`	`Subtyping.lean`
Evaluation equivalence	`concEval_equiv`	`Soundness/ConcEvalPreservation.lean`
Preservation	`concEval_preservation`	`Soundness.lean`
Progress	`synth_progress`	`Soundness.lean`
End-to-end soundness	`soundness`	`Soundness.lean`

Table 2: Paper-to-Lean correspondence

The formalisation also includes an algorithmic decision procedure for these judgments (`evalSubst`, `subCheckSubst`, `Och.check` in `EvalSubst.lean` and `API.lean`), with partial soundness verification against the declarative `Subtype'` relation.

6.3 De Bruijn details

The formalisation's seen-set S is `List (Nat × Expr × Expr)` — each entry is depth-tagged with $|\Gamma|$ at which it was recorded, and the hypothesis rule shifts entries to the current depth on use. This bookkeeping is invisible in the named presentation: free variables carry their own names, so no shift is ever required at lookup time.

6.4 Proof status

Evaluation equivalence, preservation, and progress are **sorry-free**. The end-to-end composition is sorry-free in its own body.

Bibliography

[1] D. S. Hutchins, "Pure Subtype Systems," in *Proceedings of the 37th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, ACM, 2010, pp. 287–298. doi: 10.1145/1706299.1706334.

[2] P. Fu and A. Stump, "Self Types for Dependently Typed Lambda Encodings," in *Rewriting and Typed Lambda Calculi (RTA-TLCA)*, Springer, 2014.

[3] M. Brandt and F. Henglein, "Coinductive Axiomatization of Recursive Type Equality and Subtyping," *Fundamenta Informaticae*, vol. 33, no. 4, pp. 309–338, 1998.

[4] V. Pasquale and Á. García-Pérez, "Towards the Type Safety of Pure Subtype Systems," in *34th EACSL Annual Conference on Computer Science Logic (CSL)*, in Leibniz International Proceedings in Informatics (LIPIcs), vol. 363. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2026, pp. 37:1–37:16. doi: 10.4230/LIPIcs.CSL.2026.37.